Mapa de riesgo FEN Piura · Plan del equipo

Oct 30, 2026 · @Avril

Reto, usuario y propuesta de valor

Construimos un mapa en vivo de puntos críticos ante el FEN en Piura, donde la IA clasifica cada zona en rojo, amarillo o verde para que brigadas y autoridades atiendan primero lo más urgente.

- **Track:** Solidaridad y ayuda humanitaria ante el Fenómeno El Niño (FEN) · Mapa ciudadano de riesgo.
- **Qué pide el reto:** reporte de puntos críticos con foto y tablero priorizado.
- **Usuario principal:** brigadas y autoridades de Piura (COE distrital, Defensa Civil).
- **Dolor:** los reportes llegan dispersos (llamadas, WhatsApp) y sin una prioridad común, así que se pierde tiempo en decidir a dónde ir primero. [SUPUESTO: validar con un mentor]
- **Propuesta de valor:** la brigada registra un punto en segundos; la IA lo clasifica con criterios de la Evaluación Rápida EDAN; el mapa y la tabla muestran qué atender primero y quién ya está asegurado para evacuar.
- **Dato para el pitch:** Piura es el departamento con más población expuesta a riesgo muy alto por inundaciones: 368 199 personas en el escenario mayo–julio 2026 (CENEPRED).

Cliente: SENAMHI

A SENAMHI le vendemos lo que hoy no tiene: confirmación en campo, en vivo y georreferenciada, de si sus avisos se convirtieron en daños reales.

Datos de SENAMHI que usamos:

Dato	Uso en la app	Hoy en la demo
Caudal del río Piura en Puente Sánchez Cerro, Tambogrande y Puente Ñácara	Panel "Estado del río" y entrada a la IA: si la estación cercana está en naranja o rojo, sube la prioridad	Valores simulados con umbrales reales
Umbrales de Sánchez Cerro: amarilla 900, naranja 1290, roja 1900 m³/s (El Tiempo)	Color del panel del río	Reales

Dato	Uso en la app	Hoy en la demo
Umbrales de Tambogrande: amarilla 760, naranja 1100, roja 1600 m³/s (misma fuente)	Igual	Reales
Avisos meteorológicos por nivel (amarillo, naranja, rojo); el de muy corto plazo es descargable en shapefile (campus SENAMHI)	Capa de contexto en el mapa	Siguiente versión
Monitoreo hidrológico en la plataforma PHISIS (SENAMHI)	Fuente en vivo en producción	Siguiente versión

Su dolor:

- Necesita verificar si sus avisos acertaron: mide su eficacia con estadísticos PD, PFA y CSI (tesis DZ12). Nuestros reportes con hora y ubicación son esa verificación.
- Sus propios materiales señalan que los avisos se guardan en shapefile pero el SIG no se aprovecha más allá del nivel de aviso (campus SENAMHI).
- Tiene pocas estaciones y radares para ver dónde avanza el fenómeno (Gestión); cada brigada funciona como un observador en terreno.

Por qué nos elegirían:

Herramienta	Qué hace	Qué le falta frente a nosotros
EDAN Digital (INDECI)	App para registrar daños tras un desastre (Canal N)	Registra el daño después; no lo cruza con avisos ni caudales [SUPUESTO]
SIGRID (CENEPRED)	Escenarios y mapas de riesgo	Es planificación, no lo que pasa hoy en la calle [SUPUESTO]
PHISIS (SENAMHI)	Monitoreo de caudales	Mide el río, no el impacto en barrios [SUPUESTO]
Nosotros	Caudal + aviso + reporte de brigada, priorizado con IA en un mapa en vivo	Prototipo; falta conectar datos reales

Riesgo: SENAMHI pronostica, no responde a emergencias. Quien opera la respuesta es el COER o la municipalidad. Mejor presentarlo así: SENAMHI como cliente de los datos de verificación y COER/brigadas como usuarios diarios.

Cómo se puntúa (1000 pts)

Código y pitch suman 600 de los 1000 puntos; el envío es automático y se puede reenviar cada 5 min, así que conviene subir temprano y mejorar.

Ítem	Puntos	Qué implica para nosotros
Entregar y mejorar el código	350	Zip del proyecto con la carpeta `.kiro/` (specs, steering, hooks)
Grabar y mejorar el pitch	250	Video del pitch: reservar tiempo y guion
Registrar el frontend	150	URL pública https; un agente de navegador la recorre, toma capturas y puntúa
Subir el documento (PDF o PPTX)	100	Problema, solución, arquitectura y cómo usamos Kiro
Bonus: GenAI con propósito	40	Bedrock clasifica rojo/amarillo/verde y sugiere acción
Bonus: Asíncrono o agente	35	SQS dispara la Lambda que llama a Bedrock
Bonus: Maestría Kiro	30	Specs por funcionalidad, steering con fileMatch, hooks
Bonus: Persistencia	25	DynamoDB
Bonus: API viva	20	API Gateway respondiendo al evaluar

El bonus se acota a 150. Al registrar el frontend piden el flujo principal y la funcionalidad GenAI en 1–2 líneas:

- **Flujo principal:** registrar un punto crítico y verlo en el mapa con su color de riesgo.
- **Funcionalidad GenAI:** la IA clasifica cada punto en rojo, amarillo o verde con criterios EDAN y sugiere la acción.

Qué construimos

Una sola app para brigadas y autoridades: mapa en vivo con zonas de color, tabla al lado y un formulario de reporte.

Alcance mínimo, en orden de prioridad:

1. **Mapa en vivo (Google Maps):** cada punto es una zona circular roja, amarilla o verde; el tamaño depende de las personas en riesgo. Se actualiza cada 5 s.

2. **Tabla al lado del mapa:** ordenada rojo → amarillo → verde; muestra distrito, peligro, nivel, porqué de la IA, acción sugerida y estado.
3. **Marcar estado:** pendiente → en atención → asegurado (listo para evacuar).
4. **Formulario de reporte de la brigada:** datos EDAN mínimos y foto opcional.
5. **Clasificación con IA:** el punto aparece en gris "clasificando…" y luego toma su color.

Fuera por ahora: login, notificaciones, subida real de fotos a S3 y estadísticas. Solo se suman si sobra tiempo.

Datos del reporte y clasificación con IA

El reporte es una versión mínima de la Evaluación Rápida EDAN PERÚ, el formulario oficial que levantan las brigadas municipales y comunitarias (Plan EDAN, CARE).

Campo	Valores
Tipo de peligro	inundación, desborde de río, desborde de quebrada, colapso de dren, huaico, vivienda colapsada
Distrito y ubicación	nombre + lat/lng (clic en el mapa)
Personas en riesgo	número aproximado
Viviendas	ninguna, afectadas, inhabitables, colapsadas
Vía de acceso	libre, restringida, bloqueada
Lugares críticos cerca	colegio, posta, puente, ninguno
Necesidad urgente	evacuación, agua, albergue, atención médica, ninguna
Descripción y foto	texto libre; foto opcional

Criterios de la IA (Bedrock):

- **Rojo:** vía bloqueada con personas en riesgo, viviendas inhabitables o colapsadas, necesidad de evacuación, o colegio/posta expuestos.
- **Amarillo:** viviendas afectadas, vía restringida o riesgo que puede escalar.
- **Verde:** daño menor, sin personas en riesgo inmediato.

Devuelve nivel, porqué (máx. 15 palabras) y acción sugerida. Si Bedrock falla, estas mismas reglas clasifican por código.

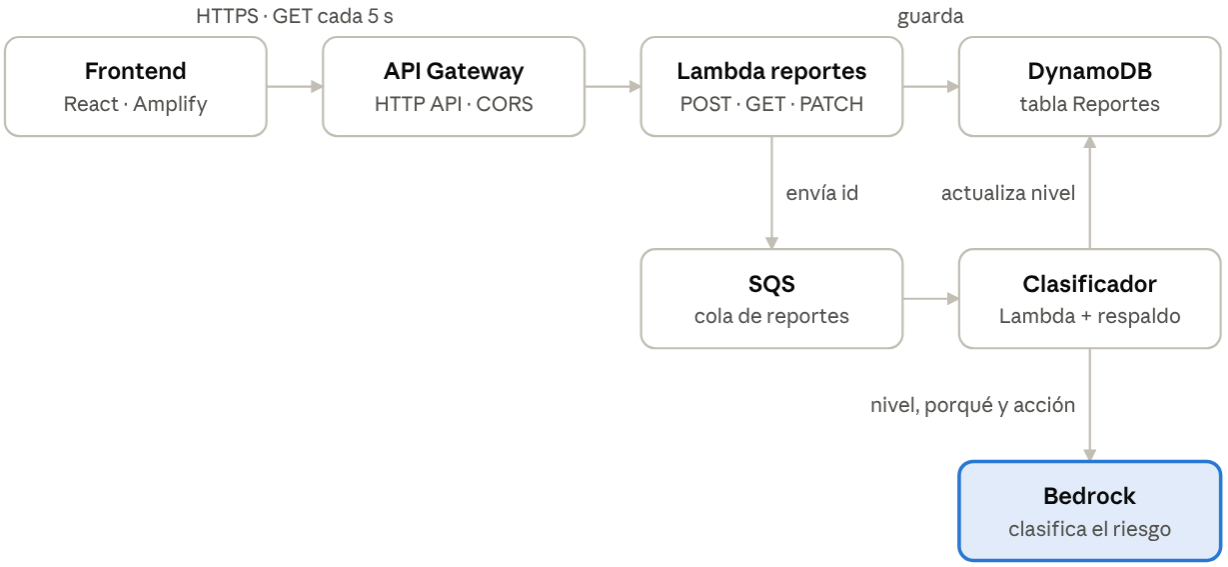
Los 3 colores equivalen a la escala de CENEPRED, que usa cuatro niveles (muy alto rojo, alto naranja, medio amarillo, bajo verde); nuestro rojo agrupa muy alto y alto (SIGRID CENEPRED).

Arquitectura y contrato de API

Serverless en AWS us-east-1: API Gateway (HTTP API) + Lambda + DynamoDB + SQS + Bedrock, con el frontend en Amplify. CORS activo desde el inicio (origen "*" en desarrollo, luego el dominio de Amplify; métodos GET, POST, PATCH, OPTIONS).

Flujo asíncrono: POST guarda el reporte con nivel "clasificando" y manda el id a SQS; responde 201 al instante. Una Lambda lee la cola, llama a Bedrock y actualiza nivel, porqué y acción sugerida.

El reporte responde al instante; la IA lo colorea después



arquitectura · 7 componentes, flujo asíncrono

El POST responde al instante; la cola desacopla la clasificación, y el punto pasa de gris a su color cuando Bedrock termina.

```
POST /reportes
  envía:  { "tipoPeligro": "desborde de quebrada", "distrito": "Castilla",
            "lat": -5.196, "lng": -80.622, "personasEnRiesgo": 40,
            "viviendas": "inhabitables", "viaAcceso": "bloqueada",
            "lugaresCriticos": ["colegio"], "necesidad": "evacuación",
            "descripcion": "El agua entra a 12 casas", "fotoUrl": null }
  recibe (201): { "id": "r-001", "nivel": "clasificando", "porque": null,
                  "accionSugerida": null, "estado": "pendiente",
                  "fecha": "2026-10-30T11:00:00Z", ...lo enviado }

GET /reportes
  recibe (200): [ objeto, ... ] ordenado rojo → amarillo → verde, luego más
reciente

PATCH /reportes/{id}
  envía:  { "estado": "en atención" }  o  { "estado": "asegurado" }
  recibe (200): objeto actualizado

nivel:  clasificando | rojo | amarillo | verde
estado: pendiente | en atención | asegurado
```

Datos de demo: 8 reportes sembrados para que el mapa nunca esté vacío (3 rojos, 3 amarillos, 2 verdes, un par asegurados) en Castilla, Piura centro, Veintiséis de Octubre, Catacaos, Sullana, Chulucanas, Tambogrande y Sechura (coordenadas aproximadas).

Frontend (Alicia)

React + Vite + Tailwind, una sola pantalla principal; mientras no exista la API se trabaja con datos simulados con la misma forma del contrato.

- **Pantalla principal:** mapa a la izquierda, tabla a la derecha; en celular, mapa arriba y tabla abajo.
- **Zonas:** círculos de Google Maps con color por nivel; gris para "clasificando". Radio calculado en el frontend según `personasEnRiesgo`.
- **En vivo:** GET /reportes cada 5 s.
- **Acciones:** botón "Nuevo reporte" (formulario), botones "En atención" y "Asegurado" en cada fila. Al marcar "Asegurado", la zona pasa a borde punteado y color atenuado; arriba de la tabla, un contador de rojos pendientes y asegurados.
- **Estados que siempre se ven bien:** cargando, vacío y error.
- **Pensado para el bot evaluador:** sin login, botones con texto claro, foto opcional y botón "Usar foto de ejemplo".

- **Google Maps:** API key con facturación activa en Google Cloud, restringida al dominio de Amplify, en `VITE_GOOGLE_MAPS_KEY`. Plan B si se traba: Leaflet + OpenStreetMap.
- **Publicación:** Amplify Hosting desde el repo; probar desde otro dispositivo antes del pitch.

Equipo, Kiro y repo

Solo una persona entra al workshop de AWS, así que el código se une en un repo de GitHub con una sola carpeta `.kiro/`.

Quién	Dónde trabaja	Qué hace
Compañero del workshop	Kiro del workshop + cuenta AWS (us-east-1, dura 36 h)	Backend, Amplify y envíos a la plataforma
Alicia	Kiro de escritorio (kiro.dev, modelo Auto)	Frontend, steering de componentes, hooks de frontend
Por definir	—	Documento PDF/PPTX y video del pitch

Repo: `frontend/`, `backend/` y `.kiro/` en la raíz. Cada uno crea su spec en `.kiro/specs/` (uno por funcionalidad) para trabajar en paralelo.

Kiro para sumar "Maestría Kiro": steering fundacional (`product.md`, `tech.md`, `structure.md`), un steering con fileMatch en `frontend/src/components/**` para el estilo visual, y hooks visibles (generar tests al guardar un componente; botón manual para el mensaje de commit).

Pendiente: nombres y roles del equipo, y la hora de cierre.

Plan y checklist

Primer envío de código y frontend a las 14:30, para mejorar con el feedback antes del cierre (hora por confirmar).

Bloque	Qué sale
11:00–11:30	Contrato acordado, repo, steering y API key de Google Maps
11:30–13:00	Spec del frontend y pantallas con datos simulados; backend con endpoints y Bedrock

Bloque	Qué sale
13:00–14:30	Conectar con la API real y publicar en Amplify
14:30	Primer envío de código y frontend
14:30–cierre	Documento, video del pitch, pulido y reenvíos

- [x] ~~Elegir el track~~
- [x] ~~Entrar al workshop (compañero)~~
- [x] ~~Kiro de escritorio instalado (Alicia)~~
- [ ] Contrato de API confirmado con backend + CORS
- [ ] API key de Google Maps
- [ ] Steering y spec del frontend en `.kiro/`
- [ ] Endpoints vivos + datos de demo sembrados
- [ ] Frontend publicado en Amplify y registrado
- [ ] Zip con `.kiro/` enviado
- [ ] Documento PDF/PPTX subido
- [ ] Video del pitch grabado
- [ ] Video de respaldo de la demo